Module 9: TypeScript Performance Optimization & Compilation

1. Introduction to Performance Optimization in TypeScript

TypeScript offers various techniques to optimize code execution and compilation.

2. Improving TypeScript Compilation Performance

Enable Incremental Compilation

Adding `"incremental": true` in `tsconfig.json` helps cache previous compilations.

```
json
{
"compilerOptions": {
"incremental": true
}
}
```

Use `skipLibCheck` to Speed Up Compilation

```
json
{
"compilerOptions": {
"skipLibCheck": true
}
}
```

3. Optimizing Type Checking

Avoid `any` and Use Strict Types

Using `any` disables type safety and slows down debugging.

```
typescript
let data: any = "This is unsafe"; // Avoid this
let safeData: string = "This is better"; // Use specific types
```

Use Union and Intersection Types Efficiently

```typescript
type User = { name: string; age: number };
type Admin = User & { role: string };

const admin: Admin = { name: "Alice", age: 30, role: "Manager" };
```

4. Optimizing TypeScript Code Execution

Avoid Unnecessary Type Assertions

```typescript
let input = document.querySelector("#input") as HTMLInputElement;
```

Use `readonly` for Immutable Objects

```typescript
interface Config {
readonly apiUrl: string;
}

const config: Config = { apiUrl: "https://api.example.com" };
// config.apiUrl = "https://newapi.com"; // Error: Cannot modify readonly property
```

Use `const` Over `let` Whenever Possible

```typescript
const pi = 3.14; // Faster and immutable
let radius = 10; // Mutable, use only if necessary
```

5. Tree Shaking and Dead Code Elimination

Use ES Module Imports to Enable Tree Shaking

```typescript
import { usefulFunction } from "./utils";
```

Avoid Unused Imports

```
typescript
```

```
import { unusedFunction } from "./utils"; // Remove unused imports
```

6. Using Web Workers for Heavy Computations

```
typescript
```

```
const worker = new Worker("worker.js");

worker.onmessage = (e) => {
console.log("Received:", e.data);
};

worker.postMessage("Start Processing");
```

7. Using `tsc` Compiler Flags for Optimization

```
sh
```

```
tsc --noEmitOnError --strict --incremental
```

Practice Questions:

1. How does `"incremental": true` improve TypeScript compilation speed?
2. What is the purpose of `readonly` in TypeScript?
3. Explain how tree shaking works with TypeScript.
4. Write a TypeScript function that optimizes a large computation using Web Workers.
5. Why should we avoid unnecessary type assertions?

Practice Answers:

1. `"incremental": true` caches previous compilations, reducing the need for full recompilation, thereby improving speed.
2. `readonly` ensures that object properties cannot be modified after initialization.
3. Tree shaking eliminates unused code during the bundling process, reducing file size and improving performance.
4. Web Worker Example:

```typescript
const worker = new Worker("worker.js");
worker.postMessage({ numbers: [1, 2, 3, 4, 5] });

worker.onmessage = (e) => {
console.log("Processed Data:", e.data);
};
```

5. Unnecessary type assertions can lead to incorrect assumptions about variable types, causing runtime errors.